Yocto Questions

Basic Yocto Questions (knowledge / understanding):

① What is the Yocto project and what are its core components?

⇒ The Yocto project is an open-source collaboration project.

⇒ providing, tools, metadata, processes to create a custom Linux distributions for embedded systems.

Core Components:

* BitBake – Build engine to process take and recipes
* policy – Reference distro combining BitBake + metadata
* Metadata Layers – organized recipe / configuration collections
* Recipes (.bb) – define how to build individual recipes
* classes (.bbclass) – share build logic across recipes
* Config files (.conf) – configure, machines, distro, builds
* OE-Core – Base set of metadata used by most Yocto setups.
* Bsp-layers – Hardware / platform-specific support.

② O/B ⇒ openembedded & Yocto project.

OpenEmbedded	Yocto project
* A build framework and metadata layer	* A project that uses open Embedded and adds tools/flo

* Contains core metadata : OE-core
* Developed earlier
* Community driven

* Provides policy (reference distro) and BitBake tool.
* Started later to unify embedded Linux build systems.
* backed by Linux Foundation and large vendors.

③ Role of BitBake in Yocto:

⇒ BitBake is the build engine used by Yocto to process tasks and recipes.

⇒ It parses recipes (.bb), resolves dependencies, and executes tasks like fetching, patching, compiling and packaging.

⇒ It similar to make but tailored for embedded systems and cross-compilation.

BitBake core-image-minimal -

④ What are layers in Yocto and why are they important?

⇒ Layers are modular collections of recipes, configurations, and classes.

⇒ Help organize metadata for different purposes.

⇒ promote reusability, scalability, and separation of concerns.

Eg of layers:

→ meta — (openembedded core)

⇒ meta-yocto - policy

⇒ meta-rpi - Raspberry pi

⇒ meta-yourapp custom application layer.

⑤ What is a Recipe (.bb) in Yocto? key fields:

A recipe defines how to fetch, configure, build, and ensta a software package.

⇒ Description : Summary of package

⇒ Homepage : project website

⇒ License : License type

⇒ SRC_URI : source location

⇒ DEPENDS Compile-time dependencies

⇒ RDEPENDS Runtime dependencies

⇒ do_compile(), do_install() : custom build steps.

⑥ How does Yocto handle dependies between Recipes?

⇒ Uses DEPENDS (compile-time) and RDEPENDS (runtime) to track dependencies.

⇒ BitBake automatically schedules build tasks in the correct order.

⇒ Each recipe builds after its dependencies are built.

⑦ D/B PACKAGECONFIG, DEPENDS, and RDEPENDS

PACKAGECONFIG	optional features or flags (like enable-x11).
DEPENDS	compile time dependencies
RDEPENDS	Runtime dependencies.

⑨ What are DISTRO_FEATURES and how do they affect the build?

⇒ DISTRO_FEATURES is a list of features to be included in your distribution.

⇒ Control conditional builds and package configurations.

⇒ Common values: x11, systemd, bluetooth, wifi etc.

⑨ How can you add a custom layer in Yocto?

Create Layer:

yocto-layer create mylayer.

Add it:

BBLAYERS += "path/to/meta-layers"

⑩ What are Machine Configurations in Yocto? Examples.

Define hardware-specific settings.

⇒ CPU architecture

⇒ Bootloader

⇒ Kernel options

⇒ Serial console

⇒ device tree.

(11) How do you add a new package to an existing Yocto image?

You can add a package to your image using the IMAGE_INSTALL variable.

[IMAGE_INSTALL += 'myapp']

Make sure:

You have a working recipe for myapp.

The layer containing that recipe is included to bblayers.conf.

(12) D/B IMAGE_INSTALL & CORE_IMAGE_EXTRA_INSTALL

variable	scope	use case.
⇒ IMAGE_INSTALL	used in image recipies	to specify what goes into that image.
⇒ CORE_IMAGE_EXTRA_INSTALL	used in local.conf (user level)	to insert packages without modifying image recipes.

(12) D/B do_configure, do_compile and do_install.

do_configure	Runs configuration scripts
do_compile	Compile source code
do_install	Installs built files into the image staging area.

(A) How to override a task (e.g., do_compile) in a Yocto recipe.

You can redefine the task inside your .bb recipes.

```
do_compile () {
    echo "Custom compile step"
    gcc main.c -o mybinary
}
```

If you want append and prepend:

```
do_compile_append () {
    echo "Extra steps after default compile"
}

do_install_prepend () {
    echo "Before install starts"
}
```

3.

(15) How to debug a recipe that fails to fetch a source tarball.

Check the logs:

bitbake <recipe_name> -c fetch -v -f

⇒ Verify SRC_URI: Ensure URL is correct.

⇒ Check proxies / firewalls.

⇒ Try manual download: Test if the link is reachable with wget.

⇒ [Export BBDEBUG = 1]

(16) How Yocto Handles source Mirrors and Local File storage

⇒ DL_DIR : All downloads go here

⇒ BB_NO_NETWORK = "1" : Forces BitBake to use only local sources.

⇒ PREMIRRORS / MIRRORS:

PREMIRRORS: checked Before trying original URL

MIRRORS : checked after original URL fails.

(17) How to fetch source from a specific git Branch or Commit:

Use SRC_URI with branch and SRCREV.

SRC_URI = "git://github.com/example/app.git;branch=main"

SRCREV = "abcdef123456..."

(18) What is devtool and How can it help during development?

devtool is a powerful Yocto CLI tool that helps

modify existing recipies

Add new recipes

Build/test your changes easily.

Eg:

devtool modify myapp

devtool modify myapp

devtool deploy-target myapp root@target

devtool finish mylayer myapp.

(19) How to apply a patch to a recipe in Yocto

place patch file inside the recipe's folder

recipes-example /myapp/myappl

update SRC_URI:

SRC_URI += "file://fix-bug.patch"

Yocto will auto-apply it during do_patch.

(20) Explain inherit in Yocto and give examples:

The inherit keyword brings in predefined classes (.bbclass) with common logic

inherit	purpose
autotools	Handles ./configure & make builds
cmake	For cmake-based projects
systemd	For systemd unit file handling
update-rc.d	For sysV init scripts
pkg-config	Enables pkg-config support.

Eg:

inherit autotools pkg-config systemd.

(21) How would you create a custom image with only specific packages?

⇒ Create a image recipe:

mkdir -p meta-myimage/recipes-core/images

⇒ write my-custom image.bb

Description = "My custom minimal image"

LICENSE = "MIT"

IMAGE_INSTALL = "busybox myapp openssh"

inherit core-image

⇒ Add to your build:

bitbake my-custom-image

(22) How do you build and integrate a custom kernel in Yocto?

Steps:

1. create a kernel recipe:

meta-myboard/recipes-kernel/linux/linux-myboard_%.bbappend.

2. Add:

SRC_URI = "git ..... branch = mybranch"

SRCREV = "commit hash"

3. set in machine conf:

PREFERRED PROVIDER_virtual/kernel = "linux-myboa

4. build it:

bitbake

(23) Explain how you could add support for a new hardware board in Yocto:

1. create a new bsp layer:

   yocto-layer create meta-myboard.

2. add machine.conf:

   meta-myboard/conf/machine/myboard.conf

3. Specify:

   MACHINE_FEATURES += "ethernet wifi"

   serial-console = "115200 ttyS0"

4. Add bootloader and kernel recipes

5. Set MACHINE = 'myboard' in Local.conf.

(24) How do you generate an SDK for application development using Yocto?

1. After buildings the image:

   bitbake my-custom-image -c populate_sdk

2. It creates:

   tmp/deploy/sdk/poky-.....toolchain.sh.

3. Install SDK:

   ./poky ... toolchain.sh.

(25) what is EXTRA_OECONF and how is it used?

Used to pass extra flags to ./configure

eg:

EXTRA_OECONF += "--enable-ssl --with-libdir=/lib64"

Internally passed like:

./configure --enable-ssl ...

(26) How does Yocto manage root file system generation and compression?

⇒ Yocto collects all files defined in IMAGE_INSTALL.

⇒ packages are staged in WORKDIR/image/.

⇒ Root filesystem types: ext4, squashfs, cpio.gz, tar.bz2.

IMAGE_FSTYPES = "tar.gz ext4".

(27) Explain the purpose of tmp/ and sstate-cache/ directories:

tmp/ — temporary build artifacts, final images, logs, binaries.

sstate-cache/ — cached prebuilt items (like object files) you reuse.

(28) How do you reduce the image size in Yocto?

Use `core-image-minimal` as above.

Remove unused packages:

IMAGE_INSTALL:remove = "python3"

Use compressed rootfs.

strip binaries:

INHERIT += "rm_work";

disable debug symbols:

INHERIT_remove = "debug-tweaks";

(29) Describe the steps to include a new init system into your image.

1. Add to DISTRO_FEATURES:

DISTRO_FEATURES += "systemd"

2. set preferred_init:

INIT_MANAGER = "systemd";

3. Ensure systemd-compatible packages are installed.

IMAGE_INSTALL += "systemd systemd-analyze"

4. In recipes:

inherit systemd

SYSTEMD_SERVICE_${PN} = "myservice.service"

(30) What are the ways to implement Over the air (OTA) updates in Yocto-based systems?

Method	descriptions
RAUC	A/B rootfs update mechanism using signed bundles.
swupdate	Flexible updater supporting multiple formats.
Mender	End to End OTA with rollback and dashboard.
ostree	Git-like filesystem update mechanism.

(31) How do you use bitbake -e and what insights can it give?

bitbake -e <recipename>

⇒ Dumps the full environment for a recipe after all .conf, .bb, .bbclass, and layer overrides are applied.

⇒ Useful to inspect:

* Final value of variables (e.g, SRC_URI, DEPENDS)
* Inheritance (inherit)
* Overrides (_append, remove etc)..

Eg:

bitbake -e busybox | grep ^SRC_URI =

(32) How do you diagnose issues in a slow yocto build?

⇒ 1. Use: bitbake -n -DDD <taeget>:

✓ Dry run with high debug level to analyse task dependencies.

⇒ 2. Enable build timing report:

BB_TASK_TIMESTATS = "1"

This creates: tmp/log/task_stats.log

⇒ 3. check parallism settings:

BB_NUMBER_THREADS = "8"

PARALLEL_MAKE = "-j8"

⇒ 4. Review dependency chain:

* Overuse of do_configure[nostamp] = "1" or disable caching.

⇒ 5. Watch disk I/O and sstate cache hits/misses.

(38) what causes a bitbake task to re-run when you didn't change anything?

- timestamp change — source file timestamps updated (even if content didn't change).
- do_compile[nostamp] = "1" — forces re-execution of the task
- using invalid sstate_cache — cache may force rebuild
- changing local.conf or env — affects the hash causing bitbake to think something changed.
- variable not whitelisted — changing unrelated var can affect recipe hash

use → bitbake -S none or -S printdiff to inspect recipe hashes.

(39) How do you cache and reuse build artifacts across team/machines?

Use shared sstate-cache + DL_DIR.

1. Host a shared cache server.

   <yocto>/sstate-cache/

   <yocto>/downloads/

2. Configure local.conf on all team machines

   SSTATE_MIRRORS = "file://.* http://..../.../PATH"

   SOURCE_MIRROR_URL = "http://build-srv/downloads/"

3. set:

   BB_HASHSERVE = "auto"

This drastically reduces build time across CI or large teams.

(35) Explain how BB_NO_NETWORK and BB_HASHBASE_WHITELIST help debugging.

BB_NO_NETWORK = "1"

Purpose:

⇒ Ensures build never accesses the internet

⇒ Helps catch recipes that silently fetch during build

→ Ensures reproducibility and integrity

BB_HASHBASE_WHITELIST

Used to ignore environment variables that would otherwise change the build hash and cause rebuilds.

BB_HASHBASE_WHITELIST += "USER DL_DIR HOME"

Use case:

⇒ Prevents bitbake from thinking the build changed just because a non-critical variable changed.

(36) If a package is not appearing in your final rootfs image, how would you troubleshoot it?

1. Check IMAGE_INSTALL:

Is the package listed in IMAGE_INSTALL or CORE_IMAGE_EXTRA_

2. Confirm package exists:

bitbake -s | grep <packagename>

3. Inspect build logs:

Was it built successfully?

bitbake <package name> -c compile -f

4. Inspect final image contents:

bitbake -c rootfs <image-name>

5. check output
   tmp/deploy/images/boards/rootfs.tar.gz

6. verify RDEPENDS/package variables
   bitbake -e <recipe> | grep ^PACKAGES=

(7) Your image boots but network does not come up - how do you debug?

1. check if connman, systemd-networkd, or ifupdown is installed
   Is your init system managing the network?

2. verify interface is up:
   ip link show

3. check logs:
   dmesg | grep eth
   journalctl -u network

4. manually assign ip:
   ifconfig eth0 up
   dhclient eth0

5. verify config files
   missing /etc/network/interfaces or .network files

6. missing firmware/drivers?
   * Inspect dmesg for missing modules.

(8) d/b prebuilt SDK vs building from yocto tree:

Aspect	prebuilt SDK	Built from Yocto tree
Setup time	Quick (one script to install)	Long (requires full yocto build)
Use case	App development outside build system	Full image customization
Performance	fast for compiling Apps	Slow, full builds

Flexibility	Limited to SDK contents	Full control (kernel, rootfs, packages, etc.)
Toolchain availability	Fixed version in SDK	Can be regenerated any

(39) How do you handle license compliance in a yocto-built system?

1. Enable license tracking:

```
INHERIT += "license"
LICENSE_CREATE_PACKAGE = "1"
```

2. Specify allowed licenses:

```
INHERIT += "license"
LICENSE_FLAGS_ACCEPTED += "commercial",
```

3. Generate license manifest:

```
tmp/deploy/licenses/
```

4. Review license files

Add them to image or deploy separately for audit

5. License warning during build.

Yocto warns if you use an unknown or disallowed license.

Use `oe-pkgdata-util list-pkgs` to verify license of each package.

(40) How do you generate and deploy signed images in yocto?

1 Signing ensures authenticity and integrity of images

A. Signed kernel / U-Boot:

Modify kernel recipe:

```
do_deploy_append() {
    openssl dgst -sha256 -sign privkey.pem
    -out ${DEPLOYDIR}/boot/zImage.
```

B. RAUC Image signing :

use meta-rauc layer.

add certificates keys to RAUC_KEY_FILE , RAUC_CERT_FILE.

IMAGE_CLASSES += "rauc"

C. ROOTFS signing:

use SIGN_IMAGE = "1":

INHERIT += "sign_image"

SIGN_IMAGE_CMD = "gpg --detach-sign ... "

can also sign UBI, ext4, or for images in do_image_complete.

(41) How do you create a dual-partition setup for A/B OTA with Yocto?

Goal: Have two root filesystems (A & B) so that an update can be applied to inactive partition and booted on next restart.

create a custom .wic file that defines two rootfs partitions:

part /boot --source bootimg-partition --ondisk mmcblk0 ...

label boot --size 64M

"

"

modify bootloader (u-boot) to support switching between A/B partitions.

=) use environment variables like boot_part = A and update logic using U-Boot scripting.

* Control active rootfs:

= you can use a bootloader.env file or write to U-Boot environment from userspace.

4. Use OTA Logic:

⇒ update rootfs while booting from rootfs A

⇒ on successful update, switch bootpart.

Tools RAUC, SW update.

(42) How do you include and configure U-Boot for a custom board?

1. Add or create a new machine layer (eg. meta-my)
2. Specify the bootloader in your machine config (meta-myboard/conf/machine/myb.conf).

PREFERRED_PROVIDER_u-boot = "u-boot-myboard"

UBOOT_MACHINE = "myboard_defconfig"

3. Add custom U-Boot recipe:

recipes-bsp/u-boot/-myb.bbappend

SRC_URI += "file://myb_defconfig"

4. Include custom patches if needed.

SRC_URI += "file://0001-fix-boot.patch"

5. Build and deploy:

bitbake u-boot.

(43) What is the role of .wic file and how do you customize them?

.wic file defines the layout of the final disk image.

Usage:

Create boot, rootfs, data partitions

Define file system types (ext4)

Install bootloaders like U-Boot.

To customize:

create your .wic in meta-your layer/wic/

point your build to it.

WKS_FILE = "custom-image.wks"

IMAGE_FSTYPES += "wic"

⑭ How would you integrate TPM or Secure Boot into a Yocto-based system?

Secure Boot:

⇒ Depends on your hardware platform

⇒ Typically involves:

- Signing U-Boot and kernel images
- validating signatures in BootROM or U-Boot

For U-Boot secure Boot:

1. Enable CONFIG_SECURE_BOOT or CONFIG_FIT_SIGNATURE

2. Signed kernel using openssl and include .dtb, kernel, initramfs in FIT image.

3. Load and verify FIT from U-Boot

TPM-Integration:

Add TPM libraries:

IMAGE_INSTALL:append = "tpm2-tss tpm2-tools tpm2-abrmd"

⑮ How do you set default system services using systemd in Yocto?

Use SYSTEMD_AUTO_ENABLE in the recipe:

SYSTEMD_SERVICE:${PN} = "myservice.service"

SYSTEMD_AUTO_ENABLE:${PN} = "enable".

Install service file:

FILES : ${PN} += "${systemd_system_unitdir}/myservice.service"

3. In your image recipe or local.conf.

IMAGE_INSTALL : append = "mypackage"

DISTRO_FEATURES : append = "systemd"

VIRTUAL-RUNTIME_init_manager = "systemd"

(46) How do you integrate third-party binary blobs into a Yocto image?

1. place binary blobs in your layer.

2. Create a recipe .bb;

DESCRIPTION = "GPU Firmware Blob"

LICENSE = "proprietary"

LIC_FILES_CHKSUM = "file://LICENSE;md5=<md5sum>"

SRC_URI = "file://gpu_fw.bin"

do_install() {

install -d ${D}${nonarch_base_libdir}/firmware

install -m 0644 ${WORKDIR}/gpu_fw.bin ${D}${non-arch_base_libdir}/firmware/

}

2. Make it as Machine-specific if needed:

PACKAGE_ARCH = "${MACHINE_ARCH}"

4. Include it in the image:

Add to IMAGE_INSTALL or create a PACKAGEGROUP.

(47) How do you test a Yocto-built image using QEMU?

1. Build a QEMU image for a supported machine;

MACHINE = qemux86 bitbake core-image-minimal.

2. Run the image using qemu.

runqemu qemux86.

3. for debugging

⇒ use serial terminal: runqemu serial stdio

⇒ Enable networking: runqemu slirp

⇒ SSH into image if network-enabled

⇒ Allows rapid testing of boot behaviour, /sources init, etc.

(Q7) How do you cross-compile a C++ application outside the Yocto tree but link it to the rootfs?

1. Generate SDK:

bitbake core-image-minimal -c populate_sdk.

2. Install SDK on your host:

./poky-glibc-x86_64-core-image-minimal-aarch64-toolchain.sh

Source env:

source /opt/poky/2.6.1/env-setup-aarch64-poky-linux.

Build you application with the cross-compiler:

$CXX hello.cpp -o hello.

Deploy binary to rootfs or device.

(Q8) What would be your approach to create a container-based Yocto system?

o/p = 1

Include docker or podman in image:

add to IMAGE_INSTALL_append:

IMAGE_INSTALL_append = " docker"

O/P 2. Create Container as recipes:
use meta-virtualization layer.
Create Docker image using dockerfile bbclass.
O/P 3
Build and deploy container post image.
use yocto just for base image, run docker at runtime to fetch containers.

Use cases:
=> Isolated app development
=> Easy system updates via Containers
=> Faster development turnaround.

(50) How do you maintain multiple products and hardware variants using the same Yocto tree?

1. Use multiple machine configurations:
=> MACHINE ?= "my1"
=> Have conf/machine/my1.conf, my2.conf etc.
2. Use DISTRO_FEATURES, PACKAGECONFIG for feature toggling
3. Use conditional recipes and package groups.
IMAGE_INSTALL_append_my1 = "my-features";
4. Shared a BSP Layers + product-specific Layers.
meta-common/
meta-productA/
meta-productB/
5. Use BBLAYERS in bblayers.conf to enable only relevant Layers.
6. Create different build dirs or local.conf files

Parsing & Internals (Google-style):-

Q. What happens internally when you run bitbake core-image-minimal?

Running bitbake core-image-minimal

Parse recipes (.bb, .bbclass, conf): Bitbake parses all metadata in meta/meta-poky/meta-yocto-bsp, meta-openembedded etc.

Resolve dependencies: bitbake analyzes all dependencies of the core-image-minimal target and its packages.

Create a task execution graph: tasks like do_fetch, do_unpack, do_compile, do_install, do_package, and do_rootfs are scheduled in dependency order.

Execute tasks: bitbake executes all tasks in the dependency graph, checking sstate-cache to skip previously built components.

Generate image. After packaging all dependencies the do_rootfs task builds the root filesystem, and do_image produces the final output

Q) How does bitbake determine whether a task needs to be rebuilt?

Bitbake uses hashes to determine whether a task needs to be rerun:

Each task has a hash (signature) generated from,

=) Recipe metadata

=) Task dependencies

=) Environment variables

=) If the hash changes compared to the sstate-cache, bitbake rebuilds it.

=) The bitbake-diffsigs tool helps you compare two hashes and shows what caused the rebuild.

③ Explain how sstate cache works, what are its advantages and failure modes?

Sstate cache: Stores build artifacts to speed up rebuilds.

=> Tasks like. do_compile, do_install and do_package produce, eg: archives in sstate-cache/.

=> When a task runs, bitbake checks the sstate hash:

If the same, it restores the task from cache instead of rebuilding.

If different, it rebuilds and updates the cache.

Advantages

=> Speeds up builds significantly,

=> Enables sharing builds across teams.

Failure mode:

=> Hash mismatches due to non-whitelisted variables

=> Corrupted cache or permission issues.

=> Environment differences across developers/machines.

④ What is BB_HASHBASE_WHITELIST used for?

This variables specifies environmental variables that are ignored when computing task hashes.

To avoid unnecessary rebuilds due to non functional changes like timestamps, paths or developer usernames,

BB_HASHBASE_WHITELIST = "DATE TIME USER"

Without this, even changing your terminal or build path might cause a full rebuild.

[How] does Yocto implement reproducible builds?

Yocto supports reproducible builds using:

[illegible]: ensures repeatable outputs.

SOURCE_DATE_EPOCH: used to eliminate time-based differences

reproducible clean binaries: timestamps and paths are deterministic

[stripping] paths in build logs and binaries using debug-prefix-map

[isolating] builds using controlled build environments (buildtools-tarball, buildtools, extended).

→ stable source inputs

→ pinned compiler versions

→ deterministic build scripts.

How do bitbake -e and bitbake -g help in understanding build internals?

bitbake -e <target>

Dumps the full environment and variable values used during build.

helps debug issues related to variables inheritance, overrides, or conflicts.

[bitbake -e core-image-minimal | grep ^EXTRA_IMAGE_FEATURES]

generates .dot files that show task and dependency graphs

outputs like:

→ task-depends.dot : task-level dependencies

→ pn-depends.dot : Recipe-level dependencies.

xdot or Graphviz.

⑦ A package builds correctly but is missing but is missing in the rootfs. How do you debug this?

Check INSANE_SKIP:

bitbake -e your-image | grep ^INSANE_SKIP=

Verify the RDEPENDS:

=> If the package is a runtime dependency of another package, ensure it is RDEPENDS_${PN}.

Was it split into subpackages?

-dev -dbg, etc.

bitbake -e <recipe> | grep ^PACKAGES=

Look into rootfs logs:

tmp/work.../rootfs/...

tmp/work/.../image/

tmp/work/.../temp/log.do_rootfs

oe-pkgdata-util list-pkg-files | grep your-package.

⑧. What tools do you use to trace depends or do_compile features?

=> Bitbake logs,

=> verbose mode => bitbake -v -D <recipe>

=> Run tasks manually in clean shell.

bitbake -c fetch -f <recipe>

bitbake -c compile -f <recipe>

=> Use devshell to inspect environment

bitbake -c devshell <recipe>

=> Use bitbake -e to check variable context:

SRC_URI, S, EXTRA_OEMAKE, etc.

=> check config.log => tmp/...

⑦ How do you find the origin of a variable like SRC_URI when it is overridden?

Methods:

⇒ use bitbake -e <recipe> | grep -A 20 SRC_URI

show the final value and where it was set / overridden.

⇒ use with bitbake -DD <recipe>

⇒ check layer.conf, bbappends and .inc files

⇒ use bitbake-layers show recipies and show appends.

bitbake-layers show appends.

⑧ Why might a previously successfully build fail in CI/CD with no recipe change?

1. changed source upstream

using git://, http:// or file:// change contents.

soln: use SRCREV = BB_NO_NETWORK = "1" for reproducibility.

. Missing dependencies on build host:

⇒ missing python3, ninja, gtt etc.

⇒ CI may use a different Docker image or host config.

Date/time-based logic:

If recipes uses date +%y%m%d to tag builds → output hash changes.

' Sstate cache mismatch / corruption.

→ Inconsistent MACHINE, DISTRO, or TMPDIR path.

→ can lead to rebuilds or cache misses.

' Race condition / parallelism issues.

Some builds fail under BB_NUMBER_THREADS > 1.

b. Environment pollution:

local environment variables like PYTHONPATH, LD_LIBRARY_PATH etc leaking into CI.

(11) How do you override the kernel version and add a custom patchset in Yocto?

⇒ Override kernel version:

```
SRCREV = "<your-commit-or-tag-hash>"
PV = "5.10.72" # set custom kernel version
```

⇒ Use a custom kernel source:

```
SRC_URI = "....."
```

⇒ Apply patches:

⇒ Create a recipes-kernel/linux/files directory.

⇒ Add your patch files

```
SRC_URI += "file://my-feature.patch"
```

⇒ Ensure patches apply in order:

```
FILESEXTRAPATHS:prepend := "${THISDIR}/files:"
```

⇒ Rebuild kernel and image:

```
bitbake -c cleansstate virtual/kernel
bitbake core-image-minimal.
```

(12) What is the structure of a Yocto-compatible U-Boot recipe? How do you add a boot script?

Structure of U-boot Recipe:

```
DESCRIPTION = "U-Boot bootloader"
LICENSE = "GPLv2+"
SRC_URI = "....."
          file://....
```

Add and install custom boot script:

boot.cmd.

```
setenv bootargs 'console=ttyS0,115200 root=/dev/mmcblk0p2
load mmc 0:1 ${kernel_addr_r} zImage
bootz ${kernel_addr_r} - ${fdt_addr_r}.
```

2. convert to boot.scr using mkimage shown in do_compile_append().

boot.scr:

do_install_append () {

install -m 0644 ---

}.

(3) How can you debug kernel boot logs from a QEMU image?

methods:

1) Use serial console output:

runqemu qemux86-64 nographic

2) Use journalctl after boot:

once QEMU boots to shell:

journalctl -k

3) Enable verbose kernel logging:

KERNEL_EXTRA_ARGS += "loglevel=8"

4) Log file output:

runqemu qemux86-64 serial=pty

5) For graphical mode:

runqemu qemux86-64 sdl.

(4) How do IMAGE_FEATURES, DISTRO_FEATURES and PACKAGECONFIG interact?

IMAGE_FEATURES:

Controls what goes into the image

(ssh_server-dropbear, read-only-rootfs, debug-tweaks).

1) Set in your local.conf, image recipe or distro config

2) Impacts image size functionality and security.

DISTRO_FEATURES:

Influences the entire distribution configuration.

x11, systemd, wayland, ipv6, bluetooth.

PACKAGECONFIG:

controls feature toggles within individual recipes

You can enable/disable optional dependencies or backend features,

can be influenced by DISTRO_FEATURES.

(b) Explain the layer priority and recipe override mechanism

CBBLAYERS (PREFERRED_PROVIDER. etc)

BBLAYERS:

defined in conf/bblayers.conf (the lists all enabled layers, only layers listed here are used.

Layer priority (BBFILE_PRIORITY):

Determines which recipes wins if multiple layers define the same recipe.

higher priority layers override lower ones.

default priority is 6 unless explicitly set

PREFERRED_PROVIDER:

Used to select among multiple providers for the same dependency

PREFERRED_PROVIDER_VIRTUAL/kernel = "linux-mycustom"

Override order:

⇒ Highest layer priority recipe wins.

⇒ can also use append in your layer to extend a lower layer's .bb file.

⇒ variable overrides (append, remove, prepend) allow fine-grained control.

bitbake -e

(16) How does Yocto manage init systems, and what happens if two conflicting services are enabled?

Yocto supports multiple init systems:

→ sysvinit
→ systemd
→ busybox init

managed through DISTRO_FEATURES:

Add systemd or sysvinit to DISTRO_FEATURES:

This controls which service manager is used during boot and what startup scripts are installed.

In recipies:

Services are declared using inherit systemd or inherit update-rc.d.

SYSTEMD_SERVICE_${PN} = "my-service.service";

Conflict Handling:

If both systemd and sysvinit are present.

→ you may end up installing both types of service scripts.

→ could lead to conflicting service start behaviour, non-deterministic boot, or duplicate services.

best practices: enforce only one init system using DISTRO_FEATURES.

Debugging Conflicts:

→ Use bitbake -e to check for DISTRO_Features resolution.

look into image /etc/init.d/ etc./systemd/ and etc/init.d/.

(17) How do you reduce build time in a Yocto CI setup across multiple branches?

To reduce build time:

⇒ Use shared state-cache across branches

⇒ Enable BB_SIGNATURE_HANDLER with hashing to avoid unnecessary rebuilds.

⇒ Enable BB_HASHSERVE or use a remote hash server to share task signature.

⇒ parallel builds with BB_NUMBER_THREADS and PARALLEL_MAKE

⇒ Use Cooker memory optimization (INHERIT += rm_work) after image build.

⇒ Use DL_DIR, TMPDIR, and SSTATE_DIR in shared or cached locations.

⇒ Only clean what's needed.

(18) How do you test and simulate a new BSP using Qemu before porting to hardware?

1. Add & configure a qemu-machine in your layer
2. Create/modify a custom BSP Layer with correct kernel, bootloader and machine config.

local.conf:

```
MACHINE = "qemu-arm"
```

Build Qemu-compatible image:

```
bitbake core-image-minimal
runqemu qemuarm
```

Use runqemu with arguments.

(19) How do you handle host contamination and ensure isolated builds?

⇒ Use buildtools - tarball or poky's nativesdk- environment SDK for consistent host tooling.

⇒ Always build in Docker/podman containers or dedicated build VMs

⇒ Installed de-depends.

Use INHERIT += "uninative"

for native binaries and avoid system dependency mismatches

⇒ Always clean TMPDIR before branch switch or major version change.

run yocto-check-layer (oe-check-sstate (or oe-selftest to validate setup.

(20) In what scenarios would you use a multiconfig setup?

Use multiconfig (conf/multiconfig)

⇒ You need to build multiple targets.

⇒ You want both host and target applications

⇒ You're building a firmware + bootloader in parallel but independently.

⇒ You want different configuration of same product.

⇒ You need side-by-side deployment of rootfs and initramfs.

bitbake -p -m

bitbake core-image-minimal -m my-custom-config.